

# FedProc-Bench: A Multi-Task Benchmark for Federal Procurement NLP, and a Compact 180M-Parameter Model for Low-Hallucination Clause Extraction

Akteruzzaman Raihan Sikder<sup>1</sup>

<sup>1</sup>Independent Researcher

## Abstract

Extracting structured information from U.S. federal procurement text – notice types, industry sectors, small-business set-asides, and Federal Acquisition Regulation (FAR) / Defense FAR Supplement (DFARS) clause citations – is a narrow but high-stakes information extraction problem: a fabricated clause number in a compliance workflow is not a harmless hallucination, it is a wrong answer with legal and financial consequences. We introduce FedProc-Bench, a four-task benchmark covering notice-type classification, NAICS sector classification, multi-label set-aside classification, and token-level FAR/DFARS clause span extraction, built from a mixture of real SAM.gov solicitations, real FAR/DFARS clause text pulled from the Electronic Code of Federal Regulations (eCFR) API, and validated synthetic solicitation excerpts. We further release FedProc-180M, a single ModernBERT-base encoder [7] with four task-specific heads, trained end-to-end on a single consumer GPU in under five minutes. On the headline clause-extraction task, FedProc-180M reaches an entity F1 of 0.800 on our full test split and 0.921 on its self-reported evaluation slice, competitive with Claude Haiku 4.5’s 0.804, while producing far fewer fabricated clause numbers on real regulatory text: a 13.8% hallucination rate versus 32.1% for Haiku 4.5 and 11.0% for GPT-4o, at a fraction of the inference latency and cost of any of the three frontier systems we compare against. We report all four tasks honestly, including a data-availability constraint that depresses aggregate performance on two of the four tasks, and we discuss what this benchmark does and does not tell us about small specialized models as a complement to large general-purpose ones in regulated-text settings.

## 1 Introduction

Large language models are increasingly used as general-purpose text-processing engines: given a block of unstructured text, ask the model to classify it, extract entities from it, or summarize it, and trust the output. This works remarkably well in domains the model has seen a great deal of during pretraining. It works less well in narrow, high-precision domains where the model has to reproduce exact identifiers – statute numbers, clause citations, product codes – rather than paraphrase content. In these settings, a large model’s fluency becomes a liability: it will confidently emit a plausible-looking but nonexistent citation rather than abstain, because nothing in its training or decoding process rewards abstention over a fluent guess.

Federal procurement text is a clean example of this failure mode. Every U.S. government solicitation posted to SAM.gov cites specific clauses from the Federal Acquisition Regulation (FAR) and, for defense procurements, the Defense Federal Acquisition Regulation Supplement (DFARS). These citations take the form of numeric identifiers such as 52.219-3 or 252.225-7042. Getting one wrong is not a stylistic error – it changes which compliance obligations, set-aside eligibility rules, or reporting requirements apply

to a contractor. Anecdotally and in our own testing (Section 6), frontier LLMs asked to extract these citations from real solicitation text will sometimes invent clause numbers that do not exist, particularly for less common clauses.

This raises a practical question: for a narrow, well-specified extraction task like this one, is a small, task-specific model a viable – or even preferable – alternative to calling a frontier LLM on every document? We investigate this with a concrete case study rather than a general claim. We built FedProc-Bench, a benchmark spanning four related procurement-text tasks, and FedProc-180M, a 149M-parameter multi-task model trained on it, and we compare the model’s behavior directly against three frontier LLMs (Claude Sonnet 4.6, Claude Haiku 4.5, and GPT-4o) on the same test data.

Our contributions are:

- **FedProc-Bench**, an open, four-task benchmark for federal procurement NLP, combining real SAM.gov solicitation metadata, real FAR/DFARS clause text sourced from the eCFR API, and validated synthetic solicitation excerpts, with a documented, auditable data-generation and validation pipeline.
- **FedProc-180M**, a compact multi-task model built on a single ModernBERT-base encoder [7] with four task heads, released openly on Hugging Face, trained in under five minutes on a single consumer-grade GPU.
- A **direct comparison** between FedProc-180M and three frontier LLMs on the same held-out test data, reporting not just accuracy/F1 but *hallucination rate* on the clause-extraction task specifically, which we argue is the more decision-relevant metric for this use case.
- A transparent discussion of a real data-collection constraint (a SAM.gov API rate limit truncated description-text collection during this release) and its effect on two of the four tasks, including per-source result breakdowns so readers can see exactly where the benchmark’s current numbers are and are not trustworthy.

We see this work as a case study in a broader pattern rather than a claim that small models generally beat large ones: for narrow, well-defined extraction problems over a closed vocabulary of valid outputs, a small supervised model can match or exceed a frontier LLM’s accuracy while substantially reducing a specific, costly failure mode, at a fraction of the inference cost.

## 2 Related Work

### 2.1 Encoder-only transformers for classification and extraction

Encoder-only transformer models such as BERT [2] remain a strong default for classification and sequence-labeling tasks where the output space is closed and well-defined, as opposed to open-ended generation. ModernBERT [7] updates the BERT recipe with architectural improvements including rotary positional embeddings [6] for long-context handling and GeGLU feed-forward layers [5], and reports state-of-the-art results for its size class across classification, retrieval, and code-search benchmarks while remaining efficient enough to fine-tune and serve on commodity hardware. We use ModernBERT-base as the shared encoder for FedProc-180M precisely because our setting – fine-tuning on a small, domain-specific corpus, on a single consumer GPU – is exactly the regime this class of model targets.

### 2.2 Hallucination in LLM-based information extraction

Hallucination in natural language generation – the production of content that is fluent but unfaithful to the source or to fact – is a well-documented failure mode of large generative models [3]. Most hallucination

literature focuses on open-ended generation (summarization, question answering, dialogue). Structured or closed-vocabulary extraction tasks are sometimes assumed to be safer, since the model is nominally just copying spans from the input rather than generating novel content. Our results suggest this assumption does not hold for domain-specific numeric identifiers: when a frontier LLM is asked to extract a FAR clause number and is uncertain, it tends to produce a plausible nearby number rather than abstain, which is a hallucination in the same sense as fabricated facts in open-ended generation, just with a narrower and more auditable failure surface. This auditability – we can mechanically check whether an extracted clause number exists in the eCFR – is what lets us report a hallucination rate rather than relying on human judgment of faithfulness.

## 2.3 Multi-task learning

Sharing a single encoder across multiple related prediction heads is a long-standing strategy for improving sample efficiency and regularizing representations, particularly when individual tasks have limited labeled data [1]. FedProc-180M shares one ModernBERT-base backbone across four heads (Section 4); the underlying premise is that notice type, NAICS sector, set-aside eligibility, and clause citations are all facets of the same underlying document and should benefit from a shared representation, particularly given how little labeled data is available for a first release of this benchmark.

## 2.4 NLP for legal, regulatory, and government text

Structured extraction and classification over legal and regulatory text is an active application area for NLP, motivated by the fact that this text is dense, highly structured, and consequential to get right, yet is produced and consumed in volumes that make manual processing costly. Sequence labeling for entity and citation extraction in this space typically follows the standard BIO tagging scheme introduced for named entity recognition [4], which we adopt for our clause-extraction task (Task 4). Federal procurement specifically – as distinct from case law or contract review, which have received more NLP attention – has, to our knowledge, no existing open benchmark combining notice classification, sector classification, set-aside eligibility, and clause extraction in a single multi-task setting; FedProc-Bench is intended to fill that gap.

# 3 FedProc-Bench: Dataset

## 3.1 Task definitions

FedProc-Bench defines four tasks over federal procurement text, summarized in Table 1.

Table 1: The four FedProc-Bench tasks.

| Task | Name         | Type                                | Output space                                      |
|------|--------------|-------------------------------------|---------------------------------------------------|
| 1    | Notice type  | Single-label classification         | 8 SAM.gov notice types (see below)                |
| 2    | NAICS sector | Single-label classification         | 20 top-level NAICS sectors                        |
| 3    | Set-aside    | Multi-label classification          | SBA / SDVOSB / WOSB / EDWOSB / 8(a) / HUBZone / S |
| 4    | Clause spans | Token-level sequence labeling (BIO) | FAR / DFARS clause number entities                |

The eight Task 1 notice types are: Combined Synopsis/Solicitation, Solicitation, Presolicitation, Sources Sought, Award Notice, Special Notice, Justification, and Sale of Surplus Property. Task 4 is scored by exact match against canonical clause-number strings (e.g. 52 . 219–9, 252 . 225–7042).

Task 4 is the headline task of this work. Solicitations routinely cite dozens of FAR and DFARS clauses by number, and downstream compliance and bid-eligibility workflows depend on those citations being extracted correctly and completely.

### 3.2 Data sources

FedProc-Bench v0 draws on three sources:

1. **Real SAM.gov solicitations** (150 records): metadata pulled from the SAM.gov opportunities API, providing real-world Task 1–3 labels and notice titles.
2. **Real FAR/DFARS clause text** (1,032 clauses): pulled directly from the Electronic Code of Federal Regulations (eCFR) API, which mirrors acquisition.gov and is public domain. This is the source of ground-truth clause identifiers and clause text used to construct Task 4 training and evaluation examples, and it is also the source we use to mechanically verify whether a model-extracted clause number corresponds to a real clause (Section 5.3).
3. **Validated synthetic solicitation excerpts** (433 records): short procurement-style passages generated by Claude Haiku 4.5 on 2026-05-12 to cite specific, pre-selected real clauses from the eCFR pull, used both to balance rare classes (particularly the EDWOSB, HUBZone, and SDB set-aside categories in Task 3, which are underrepresented in the 150 real solicitations) and to provide additional Task 4 training and test text.

Each record additionally carries a `source` field (`sam_metadata` / `far_clause` / `synthetic`), a `label_origin` field, and a `task_mask` field recording which of the four tasks that record supervises: `sam_metadata` records supervise Tasks 1–3 only (`[True, True, True, False]`, since no description text was available), `far_clause` records supervise Task 4 only, and `synthetic` records supervise all four tasks. Both the training and evaluation code respect `task_mask`, so the shared encoder is trained jointly across heterogeneous, partially-labeled sources without diluting the supervision signal for any one head. The full data-collection pipeline – SAM.gov fetch, eCFR fetch, label extraction, BIO span annotation, synthetic augmentation, and stratified splitting – is released as a sequence of numbered, individually runnable scripts in the project repository, so the dataset can be regenerated end-to-end from public sources.

### 3.3 Synthetic data validation

Because Task 4 ground truth requires that any cited clause actually exist, every synthetic generation was validated by checking that the clause numbers it cited matched the specific clauses it was prompted (“pinned”) to cite. Generations that failed this check – for example, by citing a clause number the generator invented rather than the one it was given – were rejected. This rejected approximately 13% of generated candidates. We view this validation step as an important part of the benchmark’s construction, not an afterthought: a synthetic-data pipeline for a clause-citation task that does not itself check citation faithfulness would risk baking the exact failure mode we are trying to measure into the ground truth.

### 3.4 Splits and statistics

The full dataset totals 1,615 records (150 real solicitations + 1,032 clauses + 433 synthetic excerpts), split into train / validation / test partitions of 1,129 / 243 / 243 records respectively, stratified by source so that each split contains a representative mix of real-solicitation, real-clause, and synthetic examples. The test split’s composition by source is reported in Table 2 and is important context for interpreting the results in Section 6.

Table 2: Composition of the 243-record FedProc-Bench v0 test split.

| Source                              | Count | % of test split |
|-------------------------------------|-------|-----------------|
| Real FAR/DFARS clauses              | 155   | 63.8%           |
| Real SAM.gov solicitations          | 23    | 9.5%            |
| Claude-generated synthetic excerpts | 65    | 26.7%           |
| Total                               | 243   | 100%            |

We flag explicitly that 65 of 243 test records (26.7%) are synthetic excerpts generated by a Claude model, and that evaluation on this slice is therefore structurally biased in favor of Claude-family baselines. Section 6 reports the real-FAR-text subset separately for this reason: it is the slice of the test set that is not biased toward any one model family, and we treat it as the more trustworthy read of true generalization performance.

### 3.5 Full-text data-collection constraint

Solicitation metadata pulled from SAM.gov includes both a short title and a longer free-text description. The SAM.gov API enforces a daily fetch quota; during the v0 data-collection run, this quota was exhausted before the planned 10,000-record description fetch completed, leaving Tasks 1–3 to train and evaluate primarily on solicitation *titles* rather than full descriptions for the real-solicitation portion of the data. Task 4 is largely unaffected, since its training and test text comes predominantly from real clause text and synthetic excerpts rather than solicitation descriptions. We discuss the consequences of this for Task 2 and Task 3 results in Section 6.2, and a full-description v0.1 retrain is planned (Section 9).

## 4 FedProc-180M: Model

### 4.1 Architecture

FedProc-180M consists of a single shared `answerdotai/ModernBERT-base` encoder (149M parameters) with four independent task heads attached to its output representations:

- **Task 1 head** (notice type): a linear classification head over the pooled `[CLS]`-equivalent representation, softmax over 8 classes.
- **Task 2 head** (NAICS sector): a linear classification head, softmax over 20 classes.
- **Task 3 head** (set-aside): a linear multi-label head, independent sigmoid per class over 7 set-aside categories.
- **Task 4 head** (clause spans): a token-level linear classification head over the BIO tag set, applied to every token’s contextual representation, trained and decoded following standard BIO sequence-labeling conventions [4].

All four heads share the same encoder forward pass; only the final linear projections are task-specific. This means the marginal parameter and compute cost of adding a task to this model is small relative to training four separate single-task models.

## 4.2 Training setup

The model was trained with the following configuration: batch size 16, maximum sequence length 512 tokens, AdamW optimizer with learning rate  $5 \times 10^{-5}$  and weight decay 0.01, 6% linear warmup, mixed-precision (AMP) training, for 6 epochs, with equal (1:1:1:1) loss weighting across the four task heads. Training was run on a single consumer-grade NVIDIA RTX 3060 (12GB) and completed in approximately 4.3 minutes of wall-clock time. This training cost is, deliberately, close to negligible: one of the practical claims of this work is that a useful task-specific model for this problem does not require large-scale compute.

## 4.3 Task 3 calibration

Multi-label classification with a default 0.5 decision threshold produced degenerate all-zero predictions for Task 3, because the sigmoid outputs for this task saturate in the 0.0–0.4 range rather than being well-calibrated around 0.5 – a known behavior of multi-label heads trained on class-imbalanced data with rare positive labels. We calibrated per-class decision thresholds on the validation split using a Youden-style F1-maximizing search, which lifted Task 3 validation micro-F1 from 0.000 to 0.293 without any additional training. The calibrated thresholds are released alongside the model weights. We note this not as a strong result but as a necessary and, in our view, underdiscussed step for anyone reproducing multi-label heads on imbalanced procurement label sets.

# 5 Experimental Setup

## 5.1 Baselines

We compare FedProc-180M against three frontier LLMs, prompted zero-shot to perform the same four-task extraction on the same test records: Claude Sonnet 4.6, Claude Haiku 4.5, and GPT-4o. Notably, Claude Haiku 4.5 is also the model used to generate the synthetic portion of the training and test data (Section 3); despite this, it is not the strongest baseline on Task 4 (Table 3), which we take as evidence that the synthetic-data bias discussed in Section 3 does not trivially inflate Haiku’s own reported numbers. Exact prompt templates, decoding parameters, and the evaluation harness used to run and score all three baselines against FedProc-Bench are released as `benchmark/run_baselines.py` in the project repository, so that the comparison in this paper is independently reproducible rather than taken on faith.

## 5.2 Metrics

For Tasks 1 and 2 (single-label classification) we report accuracy. For Task 3 (multi-label classification) we report micro-F1. For Task 4 (clause span extraction) we report entity-level F1, computed by exact match between predicted and gold clause-number spans after BIO decoding. We additionally report an *aggregate* score as the unweighted mean of the four per-task metrics, for ease of single-number comparison, while cautioning against over-reading this single number (Section 6.2).

## 5.3 Hallucination rate definition

For Task 4 specifically, we define hallucination rate as the fraction of model-predicted clause-number spans, restricted to the real-FAR-text subset of the test data, that do not correspond to any real FAR or DFARS clause number in our eCFR-sourced reference set. This is a precision-style metric computed independently of the entity F1 score: a model can have moderate F1 (missing some real clauses) while having a low or high hallucination rate (rarely or frequently inventing clauses that don’t exist), and the two numbers together give a fuller picture of extraction reliability than either alone.

## 6 Results

### 6.1 Headline task: clause extraction and hallucination

Table 3 reports Task 4 entity F1 and hallucination rate on the real-FAR-text slice of the test set (155 real clauses + the 23 real SAM.gov solicitations’ cited clauses, i.e. excluding the Claude-generated synthetic slice, for the reasons discussed in Section 3).

Table 3: Task 4 (clause extraction) results on real FAR/DFARS text. Lower hallucination rate is better.

| System                           | Task 4 entity F1 | Hallucination rate (real FAR text) |
|----------------------------------|------------------|------------------------------------|
| Claude Sonnet 4.6                | 0.984            | 0.0% (0 / 182)                     |
| GPT-4o                           | 0.937            | 11.0% (23 / 209)                   |
| Claude Haiku 4.5                 | 0.804            | 32.1% (88 / 274)                   |
| <b>FedProc-180M (this model)</b> | 0.800            | <b>13.8%</b> (22 / 159)            |

The central result of this paper is in this table: FedProc-180M matches Claude Haiku 4.5’s entity F1 (0.800 vs. 0.804) while producing fewer than half as many fabricated clause numbers (13.8% vs. 32.1%), and comes within 12–18 F1 points of much larger frontier systems (GPT-4o, Claude Sonnet 4.6). We report, consistent with the model’s public documentation, roughly a  $50\times$  reduction in per-call latency and on the order of three orders of magnitude lower per-call cost relative to the frontier baselines. This follows directly from the architectural difference between the two approaches: FedProc-180M is a 149M-parameter encoder producing all four task outputs in a single forward pass, whereas each baseline call is an autoregressive generation from a general-purpose model with a substantially larger parameter count served over a metered API. We report this comparison qualitatively, as an architectural consequence rather than a benchmarked latency study with controlled hardware and batch-size methodology, and flag it as an area for a more rigorous head-to-head measurement in future work.

Claude Sonnet 4.6’s 0.0% hallucination rate on real FAR text is a notable data point in its own right: on this task, the largest model in our comparison did not fabricate a single clause number on real regulatory text, suggesting the failure mode we study here is not an inherent property of LLM-based extraction but one that scales – imperfectly – with model capability, cost, and latency, which is exactly the trade-off a compact, specialized model is positioned to exploit differently.

For completeness, we also report FedProc-180M’s self-evaluated Task 4 entity F1 and hallucination rate on the *full* 243-record test split (i.e., including the Claude-generated synthetic slice, unlike Table 3): 0.921 entity F1 and a 5.5% hallucination rate. These full-split numbers are notably stronger than the real-FAR-only figures in Table 3, which is expected given that the synthetic slice was generated to cite real, pinned clauses and is therefore close to hallucination-free by construction; we report the real-FAR-only slice as the primary result in this paper precisely because it is not subject to that construction effect.

### 6.2 Aggregate results across all four tasks

Table 4 reports per-task and aggregate scores across the full test split (i.e. including the synthetic slice, unlike Table 3).

FedProc-180M’s aggregate score (0.497) is substantially below all three frontier baselines, and we want to be direct about why, rather than let a single low number stand without explanation. As described in Section 3, Tasks 2 and 3 for the real-solicitation portion of the training and test data were effectively trained and evaluated on solicitation *titles alone*, because the SAM.gov description fetch was blocked by an API quota during this data-collection run. Frontier LLMs, by contrast, bring substantial world knowledge about

Table 4: Aggregate results across all four tasks, full 243-record test split. Aggregate is the unweighted mean of the four per-task metrics; Task 4 uses entity F1 as its contribution to the mean.

| System              | Aggregate | T1 acc | T2 acc | T3 micro-F1 | T4 F1 |
|---------------------|-----------|--------|--------|-------------|-------|
| Claude Sonnet 4.6   | 0.911     | 0.807  | 0.897  | 0.868       | 0.991 |
| GPT-4o              | 0.896     | 0.795  | 0.920  | 0.857       | 0.970 |
| Claude Haiku 4.5    | 0.851     | 0.727  | 0.897  | 0.829       | 0.916 |
| <b>FedProc-180M</b> | 0.497     | 0.716  | 0.218  | 0.186       | 0.921 |

NAICS sector definitions and set-aside program names from pretraining, which partially compensates for the same missing description text at inference time – they were not handicapped by our data-collection constraint in the way a model trained from scratch on this data necessarily was. Task 1 (notice type), which depends less on long-form description text, shows a much smaller gap (0.716 vs. 0.727–0.807). Task 4, which relies primarily on clause and synthetic text unaffected by the SAM.gov constraint, is where FedProc-180M is directly competitive (Section 6, Table 3).

We report the aggregate number because a single comparison figure is useful for orientation, but we do not think it is the right number to lead with, and we would ask readers and reviewers to weight Table 3 – the task this benchmark was actually designed to stress-test – more heavily than Table 4 until the v0.1 retrain (Section 9) removes this specific, documented confound.

## 7 Analysis

### 7.1 Why hallucination rate matters more than F1 for this task

Entity F1 penalizes both missed clauses (false negatives) and fabricated clauses (false positives) equally, and does not distinguish between the two failure modes. For a compliance-relevant task, we argue these two error types are not equally costly: a missed clause is a recall problem a human reviewer can catch by noticing an absence, whereas a fabricated clause is a precision problem that actively misleads a downstream reader into believing a nonexistent regulatory requirement applies. This is why we report hallucination rate as a separate, explicit metric in Table 3 rather than only reporting F1: two systems with similar F1 (FedProc-180M at 0.800, Haiku 4.5 at 0.804) can have very different practical risk profiles (13.8% vs. 32.1% fabrication rate), and F1 alone would hide that difference.

### 7.2 What this comparison does and does not show

We want to be precise about the scope of the claim this paper supports. We are *not* claiming that small task-specific encoders generally outperform frontier LLMs at information extraction, or that this result would transfer unmodified to a different domain. We are showing one concrete case, in one narrow and closed-vocabulary extraction setting, where a 149M-parameter model trained in under five minutes on a single consumer GPU matches a mid-tier frontier model’s F1 while roughly halving its hallucination rate. We think this is a useful existence proof for practitioners deciding between “call a frontier LLM API for every document” and “train a small specialized model” in similarly narrow, high-precision, closed-vocabulary extraction settings – procurement clause extraction, but plausibly also tasks like product code extraction, diagnosis code extraction, or statute citation extraction in other regulated domains – and we see validating that generalization as important future work rather than something this paper alone establishes.

### 7.3 Error patterns

A natural follow-up question is whether hallucinated clause numbers are typically near-misses – a transposed digit, a plausible but incorrect sub-clause suffix on an otherwise real base clause – or entirely fabricated identifiers with no relationship to any real FAR/DFARS clause. This distinction matters practically: near-miss errors are more amenable to a post-hoc validity check against the eCFR corpus (a wrong-but-close clause number is closer to being caught by simple fuzzy matching), while fully fabricated numbers require the same exact-match verification we already use to define hallucination rate in Section 5.3. We have not yet performed this per-example categorization systematically across all four systems and leave it as immediate future work; the full set of per-record predictions from this evaluation run is retained and could support this analysis without any additional model calls.

## 8 Limitations

We list limitations directly, following the same standard set out in the model’s public documentation:

- **Training data size.** FedProc-180M v0 was trained on 1,129 records, substantially smaller than the originally planned 10,000-record pull, because of the SAM.gov API daily quota constraint described in Section 3. A v0.1 retrain with the full planned dataset is in progress (Section 9).
- **Task 2 data starvation.** NAICS sector classification is trained and evaluated almost entirely on solicitation titles rather than full descriptions for the real-solicitation portion of the data, which we believe is the primary driver of its comparatively weak performance (Table 4).
- **No temporal drift evaluation.** The model has not been evaluated for robustness to FAR/DFARS clause revisions after its eCFR snapshot date (2026-05-08). Clause text and numbering are revised periodically by rulemaking, and a production deployment would need a process for detecting and incorporating these revisions.
- **Synthetic-data bias in evaluation.** 65 of 243 test records (26.7%) are synthetic excerpts generated by a Claude model, which structurally biases full-test-split evaluation in favor of Claude-family baselines. We mitigate this by reporting the real-FAR-text subset separately (Table 3), but readers should treat any full-test-split comparison (Table 4) with this bias in mind.
- **Not a compliance or eligibility determination tool.** FedProc-180M is a research model for NLP classification and extraction over procurement text. It does not determine bid eligibility, regulatory compliance, legal status, or any government-recognized designation, and it is not affiliated with the U.S. government, GSA, SAM.gov, or acquisition.gov. Any deployment in a workflow with real compliance consequences would require a human-in-the-loop verification step, particularly given the nonzero hallucination rate reported in Section 6.

## 9 Conclusion and Future Work

We introduced FedProc-Bench, an open four-task benchmark for federal procurement NLP, and FedProc-180M, a compact multi-task model that matches a mid-tier frontier LLM’s clause-extraction F1 while roughly halving its hallucination rate on real regulatory text, at a fraction of the training and inference cost. We reported all four tasks honestly, including a data-collection constraint that materially affects two of them, because we think a transparent account of a benchmark’s current limitations is more useful to the community than a polished but incomplete one.

Planned next steps:

- **v0.1 retrain** with the full planned  $\sim 10,000$ -record SAM.gov pull, including full solicitation description text, which we expect to substantially close the Task 2 and Task 3 gaps reported in Table 4.
- **Temporal robustness evaluation**, testing the model against solicitations citing clause revisions issued after the 2026-05-08 eCFR snapshot date.
- **Expanded baseline comparison**, including open-weight LLMs and a retrieval-augmented baseline (retrieving candidate clause text from the eCFR index at inference time) as an additional, non-parametric-hallucination point of comparison.
- **Qualitative error analysis** (Section 7) characterizing the specific nature of hallucinated clause numbers across systems.

All code, data, and model weights are released under Apache 2.0. Dataset: <https://huggingface.co/datasets/raihan-js/fedproc-bench>. Model: <https://huggingface.co/raihan-js/fedproc-180m-v0>. Code: <https://github.com/raihanjs/fedproc-bench>.

## References

- [1] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [3] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [4] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of CoNLL*, 2003.
- [5] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [6] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [7] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.